



02. MongoDB CRUD: Create, Read & Update



Inserting Multiple Documents: insertMany()

insertMany() takes an array of objects and inserts them into the collection in a single batch:

```
mongosh - insertMany
db.student.insertMany([
  { name: "Priya Verma", age: 19, marks: 88, city: "Mumbai" },
  { name: "Amit Patel", age: 22, marks: 75, city: "Delhi" },
  { name: "Neha Singh", age: 20, marks: 92, city: "Pune" }
]);
```



Reading Documents: find() vs findOne()

- **db.student.find()** : Returns a **Cursor** (pointer) to all matched documents. If filter is empty, returns all documents.
- **db.student.findOne({ name: "Amit Patel" })** : Returns the first matching single document, or **null** if no match is found.

Cursor Chaining Methods:

- **.limit(2)** → Limits output to first 2 documents.
- **.sort({ marks: -1 })** → Sorts in descending order.
- **.countDocuments()** → Returns total count of matching documents.



Updating Documents: updateOne & updateMany

Always use the **\$set** operator, otherwise MongoDB will overwrite the entire document!

```
mongosh - update with $set
// 1. Update single document
db.student.updateOne({ name: "Rahul Sharma" }, { $set: { marks: 98 } });

// 2. Update multiple documents
db.student.updateMany({ city: "Delhi" }, { $set: { state: "Delhi NCR" } });

// 3. replaceOne (replaces entire document body except _id)
db.student.replaceOne({ name: "Amit Patel" }, { name: "Amit P.", age: 23 });
```



Comparison & Logical Query Operators



Comparison Query Operators

Operator	Meaning	Code Example	Description
<code>\$eq</code>	Equal to	<code>{ age: { \$eq: 20 } }</code>	Matches values equal to 20
<code>\$ne</code>	Not equal to	<code>{ city: { \$ne: "Pune" } }</code>	Matches values not equal to Pune
<code>\$gt</code>	Greater than	<code>{ marks: { \$gt: 85 } }</code>	Marks strictly greater than 85
<code>\$gte</code>	Greater or equal	<code>{ marks: { \$gte: 90 } }</code>	Marks 90 or above
<code>\$lt</code>	Less than	<code>{ age: { \$lt: 21 } }</code>	Age strictly less than 21
<code>\$lte</code>	Less or equal	<code>{ age: { \$lte: 20 } }</code>	Age 20 or below
<code>\$in</code>	In array	<code>{ city: { \$in: ["Delhi", "Mumbai"] } }</code>	Matches any value in the array
<code>\$nin</code>	Not in array	<code>{ city: { \$nin: ["Pune"] } }</code>	Matches values not in the array



Logical Query Operators (`$or`, `$and`, `$not`, `$nor`)

```

mongosh - Logical Operators

// 1. $or: At least one condition must be true
db.student.find({
  $or: [
    { marks: { $gt: 90 } },
    { city: "Delhi" }
  ]
});

// 2. $and: Both conditions must evaluate to true
db.student.find({
  $and: [
    { age: { $gte: 20 } },
    { marks: { $gte: 80 } }
  ]
});

```



CASE SENSITIVITY TRAP: String values in MongoDB are strictly case-sensitive! "Delhi" and "delhi" do not match. Use Regex for case-insensitive searches: `{ city: /^delhi$/i }`.